scaling down to save costs, diagnosing and debugging issues, and generally, keeping the lights on. Smart operations teams try to automate as much as possible to minimize human error and free up their time so they can be proactive rather than fighting fires.

Adopting Docker changes the way we think about many of these areas.

Take configuration management, for example. Although tools like Chef, Puppet, or Ansible are extremely popular, their role in a containerized world is considerably diminished. With Docker, each piece of software has its dependencies baked into its image, isolating it from those in another container. This dramatically simplifies the problem. Since app-level dependencies are managed by the app itself, in its `Dockerfile`, configuring your infrastructure instances is largely just a case of ensuring that Docker is installed.

As we'll see shortly, Docker also has other implications, especially on how we release and operate our software.

Container Orchestration

Docker has changed the Ops landscape in two key ways. Firstly, its built-in delivery mechanism—the ability to *push* images to a Docker Registry and *pull* them down as needed—solves a common question: how do I get my software onto the target machine? Secondly, containers let us treat hugely disparate software in essentially the same way; we use the same mechanism to start, stop, and restart containers, whatever they happen to be running.

This standardization, both in terms of delivery mechanism and how we manage software, reshapes the way we think of operations. The focus shifts to how we configure, run, and update the multiple containers that make up an application *in concert*, a process known as (container) *orchestration*.

A new breed of tools has emerged to help orchestrate our app containers; they're named (rather uncreatively) *orchestrators*.

So what do these orchestrators do for us? First, they provide an environment or *platform* on which we can run and manage our containerized applications. They do this by creating an abstraction above the (physical or virtual) servers needed to run the software. When deploying applications, it's more convenient to think of our group of "compute" servers as a single logical unit: a cluster. This lets us say things like, "Let's deploy this app to the cluster," without caring how it gets deployed or on exactly which instance—we can (mostly) let our orchestration layer handle those details for us.